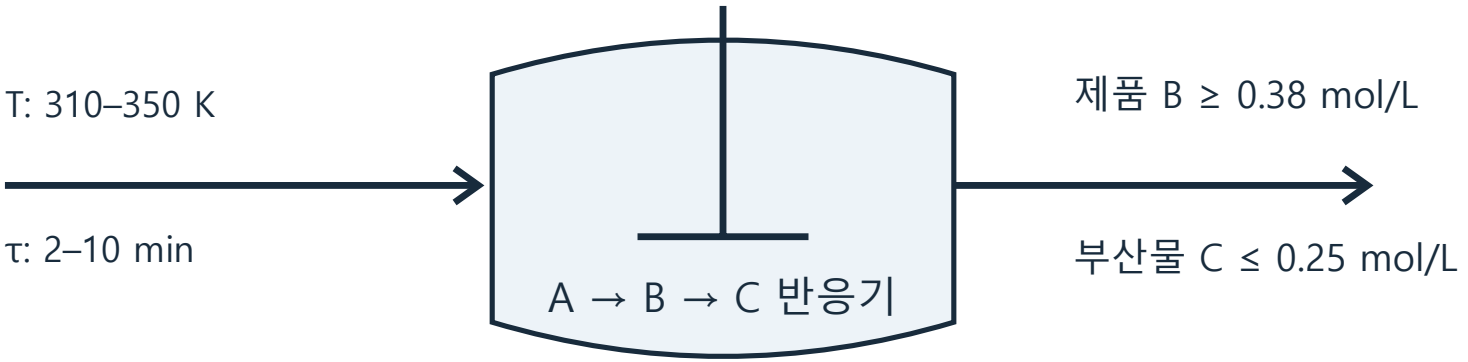
6주차: 신경망 기반 최적화의 풀이 속도와 결과 해석

제품 규격을 만족하는 반응기 조건

반응기 설계 검토에서 온도와 체류시간 후보를 반복해서 계산합니다. 제품 규격을 만족하는 조건을 찾았어도 더 나은 조건이 남아 있는지 판단해야 합니다.



계산 시간 안에 찾은 해를 어떻게 판단할까?

이번 주에는 계산을 마치기 전에도 해의 품질을 평가하고 bounds와 정식화를 조정하는 방법을 다뤄봅니다.

비교할 문제와 계산 조건

5주차 반응기 모델에 온도를 입력으로 추가합니다. 유량을 1 L/min로 고정하므로 체류시간은 반응기 부피도 결정하는 설계변수입니다.

학습을 마친 ReLU 모델, 입력 영역, 목적함수와 제품 규격을 고정하고 정식화만 비교합니다. 온도 범위를 임의로 줄이면 원래 설계 문제와 달라집니다.

모든 LP와 MILP는 오픈소스 HiGHS로 풉니다.

정식화와 solver 결과의 학습 목표

1. 가능해의 목적값과 최적값에 대한 bound를 구분하고 gap을 계산합니다.
2. 구간 전파와 LP 기반 bound tightening을 비교합니다.
3. Pyomo·OMLT에 고정 신경망과 scaling을 연결합니다.
4. 종료 상태, 계산 시간, forward 일치와 제약 잔차를 보고합니다.

5주차의 Big-M 동치성과 입력 bounds 계산을 복습한 뒤 시작합니다.

고정 신경망과 MILP 표현

각 ReLU의 입력 a에 유효한 $L \le a \le U$가 있고 $L < 0 < U$이면 다음 식에 이진변수 z를 사용합니다.

$$h \ge 0, h \ge a$$
$$h \le Uz, h \le a - L(1 - z)$$

가중치·bias·scaling은 고정합니다. 다른 제약과 목적함수까지 선형이면 전체 문제가 MILP입니다. 정확한 신경망 표현은 공정 예측 오차와 별개입니다.

가능해, bound와 gap

최소화에서 incumbent는 지금까지 찾은 가장 좋은 정수 가능해입니다. 이 가능해의 목적값을 상한으로 사용하면 유효한 하한과 함께 최적값의 범위를 정할 수 있습니다.

$$J_{\text{bound}} \le J_{\text{opt}} \le J_{\text{inc}}$$

$$g_{\text{abs}} = J_{\text{inc}} - J_{\text{bound}}, g_{\text{rel}} = \frac{J_{\text{inc}} - J_{\text{bound}}}{|J_{\text{inc}}|}$$

상한 100, 하한 80이면 절대 gap은 20이고 이 정의의 상대 gap은 20%입니다. Incumbent가 없으면 이 gap을 계산할 수 없으며, 목적값이 0에 가까우면 절대 gap을 함께 보고합니다.

Branch-and-bound의 진행

이진변수를 연속 구간으로 완화한 LP를 풀면 최소화 문제의 하한을 얻습니다. LP 해에서 분수값인 이진변수 하나를 선택해 $z = 0$과 $z = 1$인 하위 문제로 나눕니다.

정수 가능해를 찾으면 incumbent를 갱신합니다. 불가능한 노드나 하한이 incumbent보다 나쁘거나 같은 노드는 더 탐색할 필요가 없습니다.

열린 노드들의 유효한 하한을 모아 전체 하한을 계산합니다. 실제 HiGHS는 presolve, cutting plane과 heuristic도 사용하므로 원래 LP 완화값과 solver의 최종 bound는 다를 수 있습니다.

ReLU의 연속 완화

z를 [0, 1]로 완화하고 제거하면 L < 0 < U에서 다음 삼각형을 얻습니다.

$$h \geq 0, h \geq a, h \leq \frac{U}{U-L}\left(a - L\right)$$

a = 0에서 허용하는 최대 h는 −UL/(U − L)입니다. L = −1, U = 1이면 0.5지만 실제 ReLU 출력은 0입니다.

이 삼각형은 스칼라 (a, h) 그래프의 convex hull입니다. 여러 뉴런과 입력 간 상관관계를 함께 표현하는 데에는 추가 제약이 필요할 수 있습니다.

등식 제약이 있는 두 뉴런 예제

다음 무차원 예제로 bounds가 놓치는 관계를 직접 계산합니다.

$$0 \le x_1, x_2 \le 1, x_1 + x_2 = 1$$
$$h_1 = \max\left(0, x_1 + x_2 - 1\right), h_2 = \max\left(0, x_1 - x_2\right)$$
$$\min_{x_1, x_2} x_1 \text{subject to} h_1 + h_2 \ge .4$$

입력 구간만 사용하면 두 pre-activation의 bounds는 모두 [−1, 1]입니다. 하지만 첫 번째 pre-activation은 입력 등식 때문에 항상 0입니다.

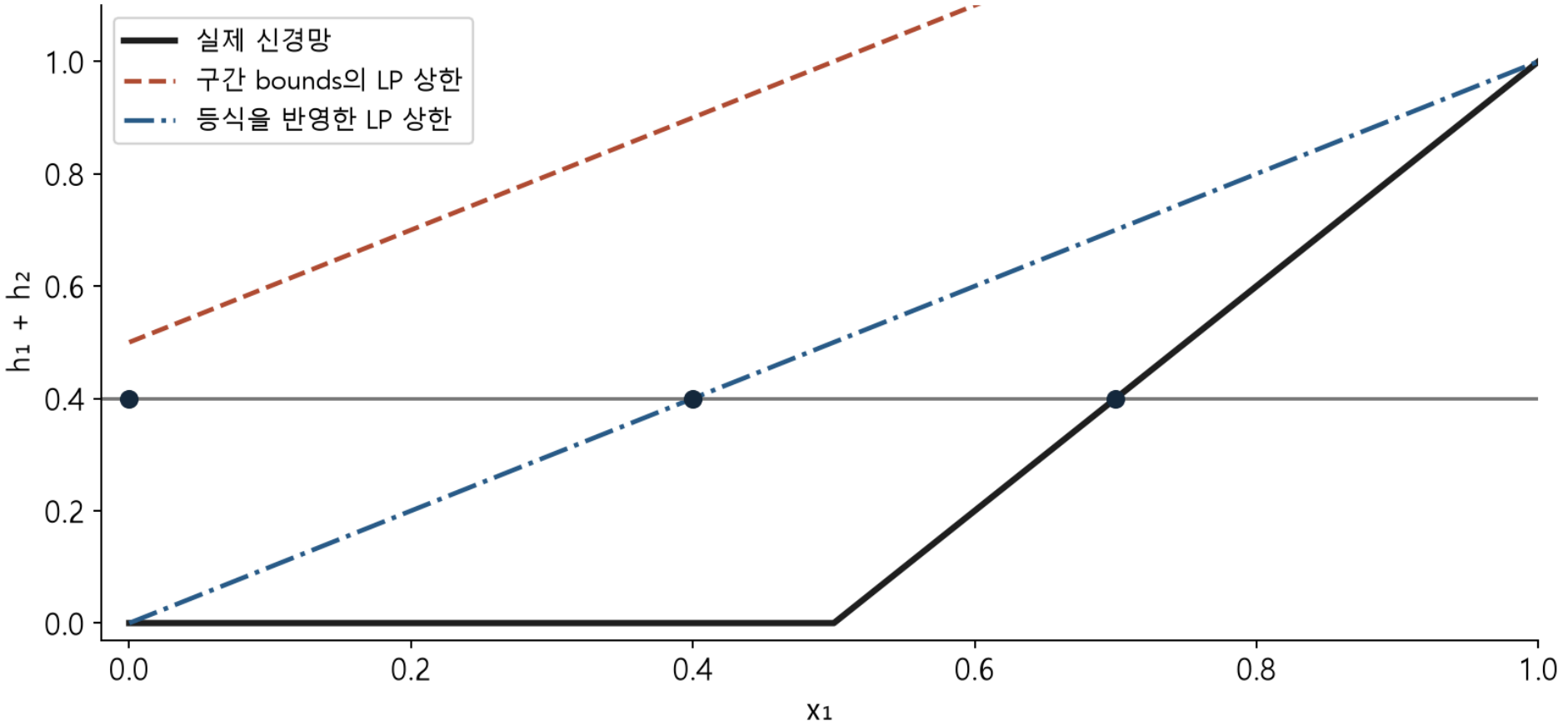
두 뉴런 예제의 정확한 해

$$h_1 = 0, h_2 = \max\left(0, 2x_1 - 1\right)$$
$$h_2 \geq .4 \Rightarrow x_1 \geq .7$$

따라서 최소 x_1은 0.7입니다. 구간 bounds를 사용하는 MILP와 등식을 반영한 MILP 모두 같은 최적값을 냅니다.

LP 완화에서는 h가 실제 신경망 출력보다 커질 수 있으므로 같은 결론이 바로 성립하지 않습니다.

Bounds 강화 전후의 LP 완화



원래 LP는 $x_1 = 0$에서도 첫 뉴런의 $h_1 = 0.4$를 허용합니다. $h_1 = 0$을 반영하면 LP 최적값은 0.4로 올라가지만 MILP 최적값 0.7에는 아직 못 미칩니다.

Bounds의 유효한 범위

Bounds는 허용한 모든 입력에서 성립해야 합니다. 학습 데이터의 최솟값·최댓값만으로는 입력 영역 전체를 덮는다고 보장할 수 없습니다.

너무 좁은 bounds는 유효한 신경망 점을 잘라내고, 넓은 bounds는 정수해의 그래프를 유지하면서 완화를 약하게 만들 수 있습니다.

제품 규격까지 사용해 계산한 bounds는 해당 규격의 가능집합에만 유효합니다. 규격이나 입력 영역을 바꿀 때는 bounds를 다시 확인합니다.

가중치 부호를 반영한 구간 전파

이전 층의 각 출력이 $\ell \le h \le u$에 있고 $w^+ = \max(w, 0)$, $w^- = \min(w, 0)$라고 놓습니다.

$$L_j = b_j + \sum_i \left(w_{ij}^{+} \ell_i + w_{ij}^{-} u_i \right)$$

$$U_j = b_j + \sum_i \left(w_{ij}^{+} u_i + w_{ij}^{-} \ell_i \right)$$

ReLU 뒤에는 [max(0, L), max(0, U)]를 다음 층에 전달합니다. 계산 비용은 작지만 각 변수의 극값을 동시에 달성할 수 있는지는 반영하지 않습니다.

LP 기반 bound tightening

P를 신경망 MILP 가능집합을 포함하는 LP 완화라고 놓습니다. 각 pre-activation을 목적함수로 바꾸어 최소화·최대화합니다.

$$L_j = \min_{v \in P} a_j\left(v\right)$$
$$U_j = \max_{v \in P} a_j\left(v\right)$$

이 방식은 optimization-based bound tightening(OBBT)입니다. 실습은 제품 규격을 제외한 입력 영역 전체에서 LP 54개를 풀고, bounds를 적용한 MILP를 새로 만듭니다.

활성이 고정된 뉴런의 처리

$$U \leq 0 \Rightarrow h = 0$$
$$L \geq 0 \Rightarrow h = a$$

이 두 경우에는 이진변수가 필요하지 않습니다. 0을 가로지르는 뉴런만 분기 대상으로 남깁니다.

실습에서 직접 작성한 MILP는 넓은 bounds에서 이진변수 24개, 구간 전파에서 22개, LP-OBBT에서 19개를 사용합니다. 이 개수는 solver presolve를 적용하기 전의 이진변수 수입니다.

입력 scaling과 물리 단위

$$x_1 = \frac{T - 310}{40}, x_2 = \frac{\tau - 2}{8}$$
$$T = 310 + 40x_1, \tau = 2 + 8x_2$$

학습 때 사용한 offset과 scale을 그대로 가져옵니다. 물리 입력의 bounds를 scaled 입력에 잘못 전달하면 신경망을 평가하는 영역이 바뀝니다.

이 실습의 출력 단위는 mol/L입니다. 표준화된 출력을 사용한다면 제품 규격에 연결하기 전에 고정된 역변환도 넣어야 합니다.

목적함수의 상수와 상대 gap

$$J = \tau + \alpha\left(T - 310\right), \alpha = .02\text{min/K}$$
$$J = 2 + 8x_2 + .8x_1$$

α는 체류시간과 온도 증가를 함께 비교하기 위한 교육용 가중치입니다. 이 목적함수의 단위는 min 등가 점수이며 공정의 실제 비용식은 아닙니다.

상수 2를 빼도 최적 입력은 같지만 상대 gap의 분모는 바뀝니다. 실습은 상수를 포함한 목적값과 bound를 그대로 보고합니다.

오픈소스 계산 도구의 역할

도구	역할
NumPy	신경망 forward 계산과 선택 재학습
Pyomo	변수·목적함수·공정 제약 작성
OMLT	고정 신경망을 Pyomo 제약으로 변환
HiGHS / highspy	LP와 MILP 풀이

모든 LP와 MILP는 MIT 라이선스의 오픈소스 HiGHS로 풉니다.

OMLT가 만든 제약도 입력 순서, scaling, bounds와 함께 확인합니다. HiGHS 공식 안내

OMLT와 공정 제약 연결

가중치와 scaling을 담은 `definition`을 만든 뒤 다음과 같이 연결합니다. `nn.inputs`는 물리 단위이고 `scaled_inputs`는 변환된 입력입니다.

```
model.nn = OmltBlock()
model.nn.build_formulation(ReluBigMFormulation(definition))
model.product = pyo.Constraint(expr=model.nn.outputs[1] >= .38)
model.byproduct = pyo.Constraint(expr=model.nn.outputs[2] <= .25)
model.obj = pyo.Objective(
    expr=model.nn.inputs[1] + .02*(model.nn.inputs[0]-310))
```

완전한 구성 코드는 실습의 `make_omlt` 함수에 있습니다.

5주차 예제의 OMLT 재현

온도 330 K, 유입 A 농도 1.2 mol/L에서 체류시간만 선택하는 기존 보간 모델을 먼저 확인합니다. $B \geq 0.40$, $C \leq 0.25$ mol/L를 적용합니다.

정식화	최소 체류시간 [min]	생성된 이진변수
직접 작성	5.353846	3
OMLT	5.353846	4

OMLT는 이 버전에서 ReLU마다 이진변수를 생성합니다. 직접 작성한 모델은 항상 활성인 뉴런 하나를 선형으로 표현하므로 이진변수가 하나 적고 출력은 같습니다.

온도와 체류시간을 선택하는 문제

$$\min_{T,\tau} J = \tau + .02\left(T - 310\right)$$
$$310 \le T \le 350, 2 \le \tau \le 10$$
$$y = f_\theta\left(T, \tau\right), y_B \ge .38, y_C \le .25$$

농도는 mol/L, 온도는 K, 체류시간은 min입니다. 각 예측 농도에는 비음수 제약도 적용합니다.

신경망은 입력 2개, 뉴런이 12개씩인 ReLU 은닉층 두 개, 선형 출력 3개로 구성됩니다. 모든 비교에 같은 고정 계수를 사용합니다.

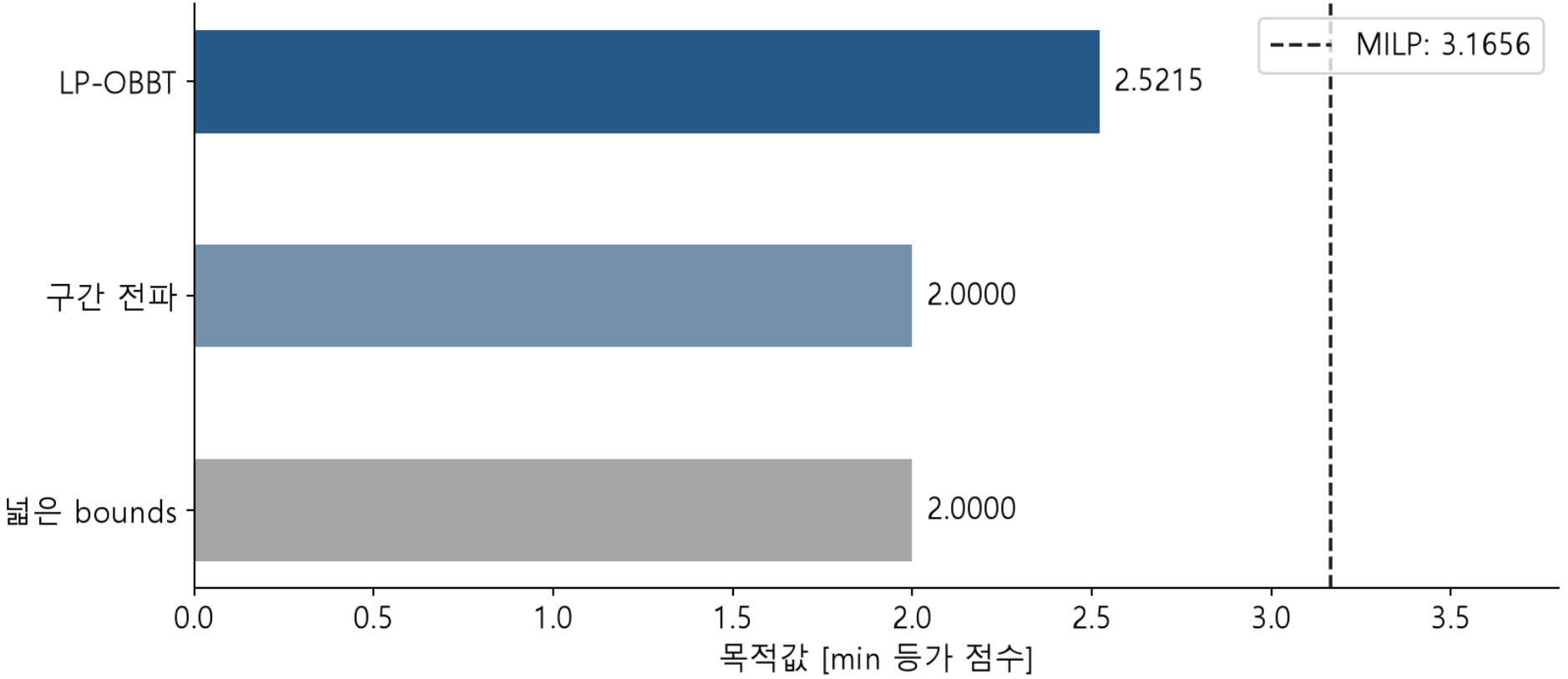
고정 모델의 학습과 독립 예측 평가

이상적 등온 A → B → C CSTR 수지에서 학습 입력 1,200개의 농도를 생성하고 NumPy Adam으로 6,000회 갱신했습니다. 독립 난수 입력 500개에서 물리 단위 오차를 평가합니다.

성분	Test RMSE [mol/L]
A	0.00546
B	0.00440
C	0.00564

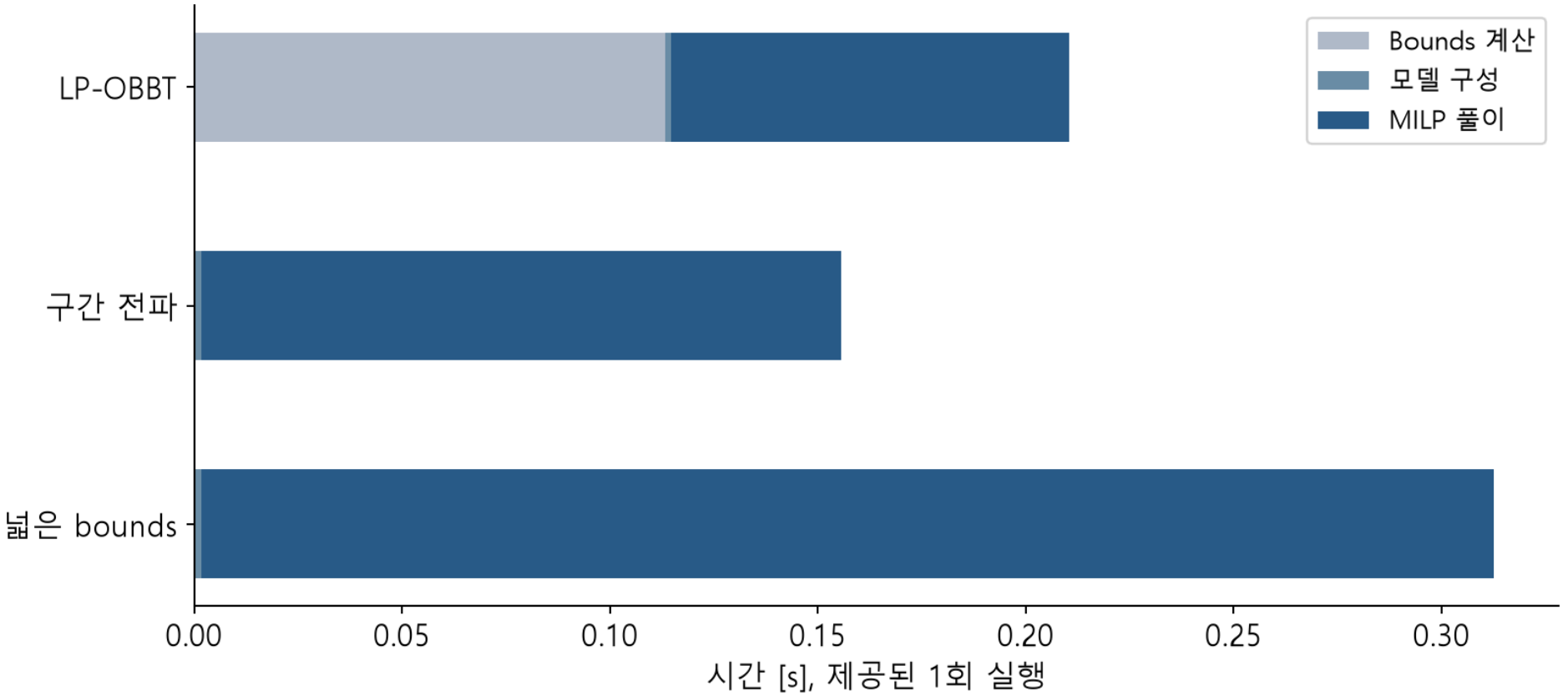
기본 실습은 제공된 계수를 읽습니다. `--refit`은 선택 재학습이며 최적화 도중 가중치를 바꾸지 않습니다.

Bounds에 따른 완화값 비교



LP-OBBT는 원래 LP 하한을 2.0000에서 2.5215로 올렸습니다. 세 MILP의 최적값은 모두 3.1656으로 일치합니다. 세 비교에는 같은 예측 모델을 사용했습니다.

Bounds 계산 비용을 포함한 시간



제공 실행에서 LP-OBBT의 MILP 풀이 시간은 줄었지만 bounds 계산을 더한 총시간은 구간 전파보다 길었습니다. 인터페이스 비용을 포함해 한 번 측정한 결과이므로 이 속도 순위가 일반적으로 성립하는지는 알 수 없습니다.

선택된 조건과 solver의 결론

항목	제공 실행
온도 [K]	342.059300
체류시간 [min]	2.524440
목적값 [min 등가]	3.165626
예측 B [mol/L]	0.380000
종료 상태 / 상대 gap	optimal / 0

HiGHS는 설정한 수치 허용오차 안에서 이 surrogate MILP의 최적성을 확인했습니다. B 하한은 활성이고 C 상한에는 여유가 있습니다.

Forward 일치와 제약 잔차 검사

제품 규격을 뺀 상태에서 입력을 고정하고 신경망 표현만 풉니다. 입력 영역의 네 모서리와 독립 내부점 12개를 직접 작성한 모델과 OMLT에 각각 넣습니다.

32회 검사에서 출력의 최대 절대 차이는 7.77e-16 mol/L였습니다. 선택된 최적 조건에서도 forward 출력을 다시 계산합니다.

제약 잔차와 이진변수의 정수성 잔차도 확인합니다. 표본 검사는 구현 오류를 찾는 데 사용합니다. 영역 전체의 수학적 동치는 5주차의 정식화 조건이 성립해야 보장됩니다.

종료 상태와 가능해 존재 여부

종료 상태	해석
optimal	설정한 gap·수치 허용오차 안에서 완료
시간 제한, incumbent 있음	가능해와 남은 gap을 보고
시간 제한, incumbent 없음	사용할 해를 아직 확보하지 못함
infeasible	작성한 문제의 가능집합이 비어 있음
unbounded / error	Bounds·목적식·로그와 모델 확인

시간 제한만으로 infeasible을 결론 내리지 않습니다. 실습의 0초 제한은 `maxTimeLimit`과 incumbent 없음, $B \geq 2$인 경우는 `infeasible`을 반환했습니다.

가능해 확인과 결과 불러오기

```
opt = Highs()
opt.config.load_solution = False
opt.config.time_limit = 30.0
result = opt.solve(model)
inc = result.best_feasible_objective
print(result.termination_condition)
if inc is not None and np.isfinite(inc):
    result.solution_loader.load_vars()
    print(inc, result.best_objective_bound)
```

가능해가 없는 종료에서 변수 값을 읽으면 초기값이나 이전 계산값을 결과로 오해할 수 있습니다. 실습 함수는 gap과 제약 잔차도 반환합니다.

Solver gap과 공정 예측 오차

선택한 같은 입력	B 농도 [mol/L]
신경망 및 MILP	0.380000
기준 CSTR	0.371157
요구 하한	0.380000

기준 CSTR의 B 농도는 규격보다 낮습니다. 정식화가 forward와 일치하고 solver gap이 0이어도 공정 모델의 제약 만족은 따로 확인해야 합니다.

8주차에서는 이런 선택 지점의 오차를 바탕으로 조건을 재검토하고 추가 데이터를 정합니다.

실습 실행과 비교 보고서

실습 묶음을 풀고 같은 폴더에서 실행합니다.

```
python -m pip install -r requirements.txt
python week06_lab.py --output-dir my-results
```

1. 두 뉴런 예제의 LP 값 0, 0.4와 MILP 값 0.7을 손으로 확인합니다.
2. 동일한 고정 모델에서 bounds·이진변수 수·LP 하한·총시간을 비교합니다.
3. 종료 상태·gap·forward 차이와 기준 CSTR의 B 규격 위반을 보고합니다.

읽기자료 · 실습 ZIP

출처와 지정 읽기

OMLT 논문: Ceccon 등, JMLR 23(349), 2022 · 공식 사용 예제

Anderson 등, Strong mixed-integer programming formulations for trained neural networks, Mathematical Programming, 2020

HiGHS 공식 사이트와 MIT 라이선스 안내 · Pyomo의 HiGHS 인터페이스

읽기자료에서 삼각형 완화 유도, OBBT의 bound 방향, scaling과 gap 계산을 따라갑니다. Anderson 등의 다변수 강한 정식화와 partition formulation은 선택 심화입니다.